

Anonymous Functions (Lambda Functions)

A **lambda function** is a small anonymous function that is **used to perform short tasks** without defining a full function using `def`.

What is a Lambda Function?

A **lambda function** is a function without a name. It can take any number of arguments but **only has one expression** (a single line of code that returns a result).

Syntax of Lambda Function

Python

```
lambda arguments: expression
```

- `lambda` – keyword to define a lambda function
- `arguments` – input values (like parameters in a normal function)
- `expression` – a single line of code that runs and returns a result

Example of Lambda Function

Python

```
# Regular function

def square(x):

    return x * x

print(square(5)) # Output: 25

# Lambda function doing the same thing

square_lambda = lambda x: x * x

print(square_lambda(5)) # Output: 25
```

Here, `lambda x: x * x` takes **one argument** `x` and returns `x * x`. It works **like the square function** but in just **one line**.

Why Use Lambda Functions?

- **Concise & Short** – No need to use `def` and `return`.
- **Used with Functions like `map()`, `filter()`, and `reduce()`.**
- **One-time use** – Good for quick operations that don't need a full function.

Lambda Function with Multiple Arguments

A lambda function can take **multiple arguments**:

```
Python
# Normal function

def add(a, b):

    return a + b

print(add(3, 5)) # Output: 8

# Lambda function doing the same thing

add_lambda = lambda a, b: a + b

print(add_lambda(3, 5)) # Output: 8
```

Lambda Function with **if-else** Conditions

A lambda function can also include an **if-else condition**:

```
Python

max_number = lambda a, b: a if a > b else b

print(max_number(10, 20)) # Output: 20
```

This means if `a` is greater than `b`, return `a`; otherwise, return `b`.

Using Lambda with `map()`, `filter()`, and `reduce()`

Using `map()` – Apply Function to Each Element

The `map()` function applies a function to **each item** in a list.

Python

```
numbers = [1, 2, 3, 4, 5]
# Square each number using map()
squared = list(map(lambda x: x * x, numbers))
print(squared) # Output: [1, 4, 9, 16, 25]
```

Using `filter()` – Keep Only True Values

The `filter()` function **filters elements** based on a condition.

Python

```
numbers = [1, 2, 3, 4, 5, 6]
# Get even numbers using filter()
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # Output: [2, 4, 6]
```

Using `reduce()` – Reduce a List to a Single Value

The `reduce()` function **reduces a list** to a single value using a function.

Python

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
# Sum of all numbers using reduce()
sum_all = reduce(lambda x, y: x + y, numbers)

print(sum_all) # Output: 15
```

Here, `reduce(lambda x, y: x + y, numbers)` works like this:

- $1 + 2 = 3$
- $3 + 3 = 6$
- $6 + 4 = 10$
- $10 + 5 = 15$

Lambda Function with Lists & Sorting

Sorting a List of Tuples

Python

```
students = [("Alice", 85), ("Bob", 92), ("Charlie", 78)]

# Sort by marks (2nd element of tuple)

sorted_students = sorted(students, key=lambda x: x[1])

print(sorted_students)

# Output: [('Charlie', 78), ('Alice', 85), ('Bob', 92)]
```

Lambda Function in Dictionaries

Sorting a Dictionary by Value

Python

```
grades = {"Alice": 85, "Bob": 92, "Charlie": 78}

sorted_grades = dict(sorted(grades.items(), key=lambda item:
item[1]))

print(sorted_grades)

# Output: {'Charlie': 78, 'Alice': 85, 'Bob': 92}
```

Lambda with Default Arguments

Python

```
greet = lambda name="Guest": f"Hello, {name}!"  
  
print(greet("John")) # Output: Hello, John!  
  
print(greet())       # Output: Hello, Guest!
```

Lambda Inside Another Function (Nested Lambda)

You can use a lambda inside another function.

Python

```
def multiply_by(n):  
    return lambda x: x * n  
  
double = multiply_by(2)  
triple = multiply_by(3)  
  
print(double(5)) # Output: 10  
print(triple(5)) # Output: 15
```

Here, `multiply_by(2)` returns `lambda x: x * 2`, so `double(5) → 5 * 2 → 10`.

Why Do We Need Lambda Functions?

Lambda functions are useful because:

1. **They make the code shorter and cleaner** – No need for long `def` functions when a simple function is required.
2. **They are useful in functional programming** – They work well with `map()`, `filter()`, and `reduce()`.
3. **They are used in Data Science and Machine Learning** – Used in data manipulation, sorting, and quick calculations.
4. **They improve readability** – When used properly, they make the code look concise and to the point.
5. **They are useful for temporary functions** – If a function is only needed once, there is no need to define it separately.

Summary Table of Operations in Lambda Functions

Operation	Example
Basic Lambda	<code>lambda x: x + 10</code>
Multiple Arguments	<code>lambda a, b: a * b</code>
If-Else Condition	<code>lambda x: "Even" if x % 2 == 0 else "Odd"</code>
With <code>map()</code>	<code>map(lambda x: x * x, list)</code>
With <code>filter()</code>	<code>filter(lambda x: x > 10, list)</code>
With <code>reduce()</code>	<code>reduce(lambda x, y: x + y, list)</code>
Sorting Tuples	<code>sorted(list, key=lambda x: x[1])</code>
Dictionary Sorting	<code>sorted(dict.items(), key=lambda x: x[1])</code>
Nested Lambda	<code>lambda x: lambda y: x + y</code>